

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms Assessment Tool 1 – Experiments 100
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	
Weightage:	40% of CIA	Total Marks:	
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Y. Jithendra Reddy	Reg. No.:	192525303 02-07-2026
Section / Batch:	AIML	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Code of Conduct

I Y. Jithendra Reddy (192525303) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Y. Jithendra Reddy

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation

Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: Signature: _____ Date: _____	Name: Signature: _____ Date: _____	Name: Signature: _____ Date: _____

Analyse Binary Search tree depth versus array-based Binary Search performance experimentally. Implement both approaches and record execution time. Interpret how tree height affects search complexity. Justify differences in performance between tree-based and array-based searches.

Algorithm: BST Depth vs Array-Based Binary Search

Aim

To compare the execution time of Binary Search Tree (BST) search and Array-Based Binary Search and study the effect of tree height on search complexity.

Algorithm 1: Array-Based Binary Search

Input: Sorted array $A[]$, key K

Steps:

1. Set $low = 0$, $high = n - 1$.
2. Repeat while $low \leq high$:
 - $mid = (low + high) / 2$
 - If $A[mid] == K$, return found.
 - If $K < A[mid]$, set $high = mid - 1$.
 - Else set $low = mid + 1$.
3. If the loop ends, return not found.

Complexity: $O(\log n)$

Algorithm 2: BST Search

Input: BST root R , key K

Steps:

1. Set $current = R$.
2. Repeat while $current \neq NULL$:
 - If $current.data == K$, return found.
 - If $K < current.data$, move to left child.
 - Else move to right child.
3. If $current == NULL$, return not found.

Complexity:

- Balanced BST $\rightarrow O(\log n)$
- Skewed BST $\rightarrow O(n)$

Experimental Procedure

1. Generate n elements.
2. Sort the elements for binary search.
3. Record start time and perform Array Binary Search.
4. Record end time and compute ArrayTime.
5. Insert the same elements into a BST.
6. Record start time and perform BST Search.
7. Record end time and compute BSTTime.
8. Compare both execution times and observe the BST height.

Result / Interpretation

- Array Binary Search is usually faster because elements are stored in contiguous memory.
- BST performance depends on tree height.
- A balanced BST gives $O(\log n)$ performance similar to binary search.
- A skewed BST increases the height and may degrade performance to $O(n)$.

Conclusion: Array-based binary search provides more consistent and efficient search performance, while BST efficiency is strongly affected by the depth (height) of the tree.

```

1 import time
2
3 # Array Binary Search
4 def binary_search(arr, key):
5     low = 0
6     high = len(arr) - 1
7
8     while low <= high:
9         mid = (low + high) // 2
10
11         if arr[mid] == key:
12             return True
13         elif arr[mid] < key:
14             low = mid + 1
15         else:
16             high = mid - 1
17
18     return False
19
20
21 # BST Node
22 class Node:
23     def __init__(self, data):
24         self.data = data

```

```

25         self.left = None
26         self.right = None
27
28
29 # Iterative BST insertion
30 def insert(root, value):
31     new_node = Node(value)
32
33     if root is None:
34         return new_node
35
36     current = root
37
38     while True:
39         if value < current.data:
40             if current.left is None:
41                 current.left = new_node
42                 break
43             current = current.left
44         else:
45             if current.right is None:
46                 current.right = new_node
47                 break
48             current = current.right
49
50     return root
51
52
53 # BST Search

```

```

54 def bst_search(root, key):
55     while root is not None:
56         if key == root.data:
57             return True
58         elif key < root.data:
59             root = root.left
60         else:
61             root = root.right
62     return False
63
64
65 # Main Program
66 n = int(input())
67 key = int(input())
68
69 arr = list(range(n))
70
71 # Build an unbalanced BST using sorted values
72 root = None
73 for value in arr:
74     root = insert(root, value)
75
76 # Array Binary Search Time
77 start = time.perf_counter()
78 array_result = binary_search(arr, key)
79 array_time = time.perf_counter() - start
80
81 # BST Search Time

```

```

11 array_time = time.perf_counter() - start
12 # BST Search Time
13 start = time.perf_counter()
14 bst_result = bst_search(root, key)
15 bst_time = time.perf_counter() - start
16
17 # For sorted insertion, BST height equals n
18 bst_height = n
19
20 print("==== Experimental Comparison ====")
21 print("Number of elements :", n)
22 print("Search key :", key)
23 print("Array search result :", array_result)
24 print("BST search result :", bst_result)
25 print("Array Binary Search Time :", array_time, "seconds")
26 print("BST Search Time :", bst_time, "seconds")
27 print("BST Height :", bst_height)
28
29 print("Conclusion:")
30 print("Array Binary Search complexity is O(log n)")
31 print("BST Search complexity depends on tree height")
32 print("Unbalanced BST search complexity is O(n)")
33

```

10000

9999

Output

```

==== Experimental Comparison
====
Number of elements : 10000
Search key : 9999
Array search result : True

```

```

BST search result : True
Array Binary Search Time :
2.1399930119514465e-05 seconds
BST Search Time :
0.0016443999484181404 seconds
BST Height : 10000

```

```

Array Binary Search complexity is
O(log n)
BST Search complexity depends on
tree height
Unbalanced BST search complexity
is O(n)

```